

FusionGuardNet: Audio Deepfake Detection

Using WavLM, Whisper, and Nes2Net

1. Introduction

1.1 Challenges in Audio-Based Deepfake Detection

Recent advances in generative AI have made it significantly easier to produce synthetic speech that sounds indistinguishable from a real human voice. Modern text-to-speech (TTS) and voice conversion (VC) systems can now generate highly convincing audio with minimal input, enabling an attacker to clone a speaker's voice from only a short recording.

These audio deepfakes pose a growing security threat. They can be used to bypass voice authentication systems, impersonate individuals in phone calls, manipulate financial transactions through social engineering, or spread disinformation through fabricated recordings. High-profile cases of voice-cloned fraud and AI-generated audio content shared on social media have already demonstrated the real-world impact of this technology, and as synthesis tools become more accessible, the scale of the threat is expected to grow.

The research community has responded to this challenge through the ASVspoof challenge series, which has run since 2015 and provides standardized benchmarks and evaluation protocols for anti-spoofing systems. Each edition introduces newer and more sophisticated spoofing systems, reflecting the ongoing arms race between generation and detection.

Building an effective detector is not straightforward. Classical approaches relied on handcrafted audio features designed for tasks like speech recognition, and they tend to miss the subtle artifacts that modern synthesis methods introduce. A deeper issue is generalization: a model trained to detect one type of fake audio may fail entirely when faced with a different synthesis method it has not seen before.

Fake audio can fail in more than one dimension simultaneously. Some artifacts are acoustic, such as unnatural spectral patterns and phase inconsistencies, while others are phonetic, such as irregular rhythm, unnatural prosody, and unusual co-articulation patterns. A detector that relies on a single feature type is therefore vulnerable to synthesis systems that do not happen to trigger that specific artifact. In practice, new generation tools appear frequently, so a detector that only works on known attack types offers limited real-world protection.

Our project addresses this challenge by exploring how combining different types of pre-trained audio representations can produce a more robust and accurate deepfake detector.

1.2 Project Goals: Improving Spoofing Classification Using Pre-trained Models

The main goal of this project is to build a system that can reliably distinguish between real and synthetic speech, and to validate it on two established public benchmarks. Rather than designing handcrafted features or training a model from scratch, we chose to leverage large pre-trained models that have already learned rich representations of natural speech from massive amounts of audio data.

Our reasoning is that these representations capture patterns in speech that are far more informative than simple acoustic features, and that a classifier built on top of them will be better equipped to detect the subtle inconsistencies introduced by fake audio.

A key design decision in this project is the use of **offline feature extraction**. Rather than running the large pre-trained encoders during each training step, we extract and save all audio representations to disk before training begins. This cleanly separates the costly encoding step from the iterative training loop, dramatically reduces GPU memory pressure, and makes it practical to experiment with different classifier architectures and hyperparameters without re-running inference through multi-hundred-million-parameter models.

Beyond using a single pre-trained model, a central goal was to explore whether combining two models with different strengths could improve detection further. Different types of synthesis errors leave different traces in the audio: some are more signal-level, while others relate to how speech is structured at a phonetic or rhythmic level. By giving the classifier a richer, multi-perspective view of the input, we aimed to improve both accuracy and robustness.

The end result is a complete, working detection pipeline that we train and evaluate across two dataset configurations. The first configuration combines two standard benchmarks: **ASVspoof 2019 LA** and **ASVspoof5 (2024)**. The second configuration extends this with a third dataset, **Fake-or-Real (for-norm)**, adding additional diversity in recording conditions and synthesis methods not present in the ASVspoof corpora. We then compare the proposed approach against single-encoder baselines.

1.3 Our Contribution: Integrating Whisper with WavLM in the Nes2Net Architecture

Our main contribution is the design and implementation of **FusionGuardNet**, a detection system that combines two pre-trained models: **WavLM** and **Whisper**. These models extract complementary representations from the same audio input.

WavLM is a self-supervised acoustic model trained on large amounts of raw speech. It captures low-level signal patterns and spectral characteristics that reflect how the voice sounds at the waveform level. Whisper, originally built for automatic speech recognition, brings a different perspective by encoding phonetic and prosodic information, capturing how speech is structured at a linguistic level.

Both representations are extracted offline, combined through a learnable weighted fusion layer, and passed into a shared **Nes2Net** classifier, which produces the final real/fake decision.

The motivation behind this design is that synthetic speech can fail in more than one way. Some artifacts are acoustic, while others are phonetic. Combining two models that analyze the signal differently gives the system a better chance of detecting both.

A further architectural choice is that the two pre-trained encoders are kept fully frozen throughout training. Only the fusion layer and the Nes2Net backbone are updated. This keeps the trainable parameter count small, prevents catastrophic forgetting of the rich representations learned during large-scale pre-training, and makes the full system practical to train in a small number of epochs on a single GPU.

We evaluate FusionGuardNet across two dataset configurations. On **Dataset 1 (ASVspoof 2019 LA + ASVspoof5 2024)**, containing 10,536 balanced test samples, the model achieves **99.18% test accuracy**, with a test loss of **0.0324** and only **86 misclassifications**. The error profile is perfectly symmetric, with equal false positive and false negative rates:

Metric	Value
Test samples	10,536
Test accuracy	99.18%
Test loss	0.0324
Total mistakes	86
True negatives (TN)	5,225
False positives (FP)	43
False negatives (FN)	43
True positives (TP)	5,225

On **Dataset 2 (ASVspoof 2019 LA + ASVspoof5 2024 + Fake-or-Real for-norm)**, containing 17,467 test samples, FusionGuardNet achieves **99.34% test accuracy** with an **Equal Error Rate (EER) of 0.60%**, demonstrating that the performance gains hold across a more diverse and modern set of spoofing conditions.

Both configurations outperform single-encoder baselines, confirming that the dual-representation fusion provides measurable added value beyond what either model achieves alone.

2. Related Work

2.1 Classical and Modern Approaches to Audio Spoofing Detection

Early anti-spoofing systems relied on handcrafted audio features combined with classical machine learning classifiers. These methods captured broad spectral properties of the audio and worked reasonably well against the synthesis systems of their time, which left relatively obvious acoustic traces.

As neural speech synthesis improved and produced more natural-sounding output, these approaches became insufficient. Their fixed representations were not sensitive enough to capture the subtle differences between real and synthetic speech.

The field gradually moved toward deep learning, which allowed models to learn more discriminative patterns directly from data rather than relying on manually designed features. This improved detection across a range of attack types.

The most recent and effective direction has been the use of large self-supervised models pretrained on vast amounts of natural speech. These models produce rich audio representations that capture general properties of human speech and have been shown to generalize better to new and unseen synthesis methods. This makes them the foundation of current state-of-the-art detection systems and the starting point for our own work.

2.2 Pre-trained Speech and ASR Models: WavLM and Whisper

WavLM

WavLM is a self-supervised speech model developed by Microsoft and trained on a large and diverse corpus of audio data. During training, it learns to reconstruct masked portions of the audio signal from noisy and clean speech, forcing it to develop a deep understanding of how natural speech is structured at the acoustic level.

It was originally designed to perform well across a wide range of speech tasks, including speaker recognition and speech separation, and achieves strong results on many of them without task-specific fine-tuning. For deepfake detection, its key value is that it captures fine-grained acoustic properties that tend to deviate from natural patterns when audio is synthetically generated.

Architecturally, WavLM is built on the wav2vec 2.0 framework, using a CNN-based waveform encoder followed by a stack of Transformer layers. The CNN feature extractor converts raw audio waveforms into a sequence of local feature vectors, which the Transformer then contextualizes across time.

What distinguishes WavLM from its predecessors is its training objective. Rather than simply masking and predicting clean speech tokens as in HuBERT or wav2vec 2.0, WavLM is trained with a masked speech prediction task applied to speech mixed with noise or overlapping utterances. This forces the model to learn representations that are robust to signal-level perturbations, a property that translates well to deepfake detection, where subtle distortions in the signal are the primary evidence of synthesis.

The variant we use, **microsoft/wavlm-base-plus**, produces frame-level embeddings of dimension **768** at a temporal resolution of one frame per **20 ms** of audio. These frame-level features preserve fine temporal structure, which is important because synthesis artifacts can appear as brief inconsistencies at specific time steps rather than as global properties of the recording.

WavLM has been shown to perform strongly on the SUPERB benchmark across a wide range of downstream tasks. Its strong speaker-discriminative properties make it especially sensitive to the acoustic fingerprint of a speaker, which voice-conversion and TTS systems must approximate but rarely replicate perfectly. In the context of anti-spoofing, WavLM representations have already demonstrated competitive performance in several prior works, reinforcing its suitability as a backbone feature extractor for our system.

Whisper

Whisper, developed by OpenAI, takes a fundamentally different approach. It is an automatic speech recognition model trained on a massive amount of transcribed audio collected from diverse sources across the internet. Because its objective is to accurately transcribe spoken content, it learns to encode phonetic and linguistic structure: how speech sounds are organized into words, syllables, and sentences.

While Whisper was not designed for deepfake detection, this type of representation is relevant. Synthetic speech often contains subtle phonetic irregularities and unnatural prosodic patterns that acoustic models may overlook. Together, WavLM and Whisper offer complementary views of the same audio signal, which is the core idea behind our fusion approach.

Architecturally, Whisper follows an encoder-decoder design. The encoder receives a log-Mel spectrogram computed from the input waveform and processes it through a stack of Transformer layers to produce a sequence of contextual embeddings. These encoder outputs are used as feature representations in our pipeline. The decoder, which generates text tokens, is discarded entirely.

The encoder's input representation, the log-Mel spectrogram, is fundamentally different from the raw waveform processed by WavLM. It captures energy distribution across frequency bins over time, reflecting how human auditory perception naturally encodes speech. The Transformer layers then attend across time and frequency to build higher-level representations of phoneme sequences, coarticulation patterns, and prosodic contours.

The variant we use, **openai/whisper-small**, also produces frame-level embeddings of dimension **768**, matching the output dimension of WavLM-base-plus. This dimensional alignment is convenient for fusion because the two feature sequences can be combined without an additional projection layer, which would introduce more parameters and potential information loss.

Because Whisper was trained across 680,000 hours of multilingual audio from highly varied recording conditions, its encoder has been exposed to a wide range of voices, accents, and speaking styles. This breadth of exposure means its encoder has developed a robust internal model of what phonetically plausible speech should look like. Deviations from this model, such as those introduced by TTS systems that imperfectly model coarticulation or rhythm, can appear as anomalous activation patterns that a downstream classifier can exploit.

2.3 Classification Architectures for Spoofing Detection: Nes2Net

To understand Nes2Net, it helps to trace the line of architectures it builds on.

Standard ResNet introduced residual connections: shortcut paths that add the input of a block directly to its output. These connections allow gradients to flow cleanly through deep networks and made it practical to train much deeper models without degradation.

Res2Net extended this idea by splitting the feature channels within a single residual block into multiple subsets and processing them in a hierarchical, sequential manner. This lets the block naturally capture features at multiple scales in a single forward pass. This multi-scale property is useful for audio because spoofing artifacts can appear at different time resolutions. A model that can attend to both local frame-level irregularities and broader temporal patterns in one block is better equipped to detect them.

Nes2Net, proposed by Liu et al. (2025), nests an additional residual connection inside each Res2Net block, creating a two-level hierarchy of residual pathways within a single building block. The inner shortcut allows the model to preserve fine-grained information, while the outer one propagates coarser, more abstract representations. This gives the architecture greater expressive capacity without requiring more depth or more parameters.

Each block also incorporates a Squeeze-and-Excitation (SE) module, which recalibrates the importance of each feature channel using global context. This further improves the model's ability to focus on the most informative parts of the representation.

A key practical advantage of Nes2Net is that it is designed to accept the high-dimensional output of speech foundation models directly, without a dimensionality reduction step. Reducing the feature dimension before classification is common in simpler pipelines but inevitably discards information. Nes2Net avoids this bottleneck.

A global pooling operation at the end of the backbone collapses the temporal dimension into a fixed-length vector, which is then passed to a linear classification head. For FusionGuardNet, this architecture serves as a compact but expressive engine that processes the fused WavLM and Whisper embeddings and produces the final real/fake decision.

3. Dataset and Preprocessing

3.1 The Selected Datasets

We conducted experiments on two dataset configurations of increasing scale. The first configuration, **Dataset 1**, combines two standard ASVspoof benchmarks. The second configuration, **Dataset 2**, extends the first by adding a third, independently sourced corpus.

The three source datasets are described below.

ASVspoof 2019 Logical Access (LA)

ASVspoof 2019 LA is one of the most widely adopted benchmarks in the speech anti-spoofing field. It contains recordings generated by 19 different text-to-speech and voice-conversion algorithms, alongside genuine human speech samples collected in a controlled recording environment.

The audio is distributed in FLAC format with accompanying protocol files that map each utterance ID to a label, either **bonafide** or **spoof**, and a source system identifier. Using this dataset allows direct comparison with a large body of prior work.

ASVspoof5 (2024)

ASVspoof5 is the most recent release in the ASVspoof series. It was collected under more challenging and diverse conditions, with a broader range of modern synthesis systems representing the current state of generative speech technology.

Including it alongside the 2019 data exposes the model to a wider variety of spoofing artifacts, which supports better generalization. It is also distributed in FLAC format with protocol files covering train and dev splits.

Fake-or-Real (for-norm subset)

Fake-or-Real is a publicly available dataset released on Kaggle, comprising real recordings and AI-generated speech samples. We use only the **for-norm** folder from this collection, which contains studio-quality, noise-normalized samples.

Unlike the ASVspoof corpora, which are organized around specific spoofing systems and protocol files, this dataset provides additional diversity in recording conditions and synthesis methods. This reduces the risk of the model overfitting to ASVspoof-specific artifacts.

The audio files are provided in WAV format and are read directly without a protocol file. Class membership is inferred from the folder structure.

3.2 Dataset Configurations

Our data organization pipeline reads the protocol files for all available ASVspoof splits and the folder structure of the Fake-or-Real subset. It then merges the entries into a unified pool before re-splitting.

Configuration	Included datasets
Dataset 1	ASVspoof 2019 LA + ASVspoof5 2024
Dataset 2	ASVspoof 2019 LA + ASVspoof5 2024 + Fake-or-Real (for-norm)

3.3 Data Splitting

After collecting all entries for a given configuration and matching each utterance ID against the available audio files, the merged pool is split into train, validation (dev), and test sets using an **80 / 10 / 10** ratio.

Class balance is enforced at the split level. Real and fake entries are shuffled independently using random seed **42** for reproducibility, and the same number of samples is drawn from each class for every split. This guarantees that no split is skewed toward either class, and that the evaluation metrics reflect genuine discriminative performance rather than class prior.

The actual dataset sizes used in our experiments were as follows:

Configuration	Split	Real	Fake	Total
Dataset 1	Train	42,139	42,139	84,278
Dataset 1	Dev	5,267	5,267	10,534
Dataset 1	Test	5,268	5,268	10,536
Dataset 2	Train	69,823	69,895	139,718
Dataset 2	Dev	8,727	8,736	17,463
Dataset 2	Test	8,729	8,738	17,467

Dataset 1 contains **105,348** samples in total. Dataset 2 contains **174,648** samples, with the additional approximately **69,300** samples contributed by the Fake-or-Real (for-norm) subset. The slight real/fake imbalance in Dataset 2 reflects the natural composition of the Fake-or-Real data, though the difference is small, less than 0.1%, and does not materially affect training.

The pipeline also produces a protocol file and a summary file for each split, recording each utterance's ID, class label, source dataset, and original split. This facilitates traceability and later analysis.

4. System Architecture and Proposed Methodology

4.1 End-to-End System Overview

FusionGuardNet follows a three-stage pipeline with clear stage separation.

Stage	Description	Output
1. Offline feature extraction	Each audio clip is preprocessed and passed through frozen WavLM and Whisper encoders.	Saved PyTorch tensor files
2. Feature fusion	The two pre-extracted feature sequences are loaded and combined by a learnable fusion module.	One fused feature sequence
3. Classification	The fused sequence is passed through the Nes2Net backbone and classified as real or fake.	Two-class prediction

Before training, each audio clip is preprocessed and passed through two frozen pre-trained encoders: WavLM and Whisper. Their output feature sequences are saved to disk as PyTorch tensor files. This happens once and is not repeated during training.

At training and inference time, the two pre-extracted feature sequences for a given clip are loaded from disk and combined by a small learnable module. The output is a single fused feature sequence of the same temporal shape as each individual input.

The fused sequence is then passed through the Nes2Net backbone, which processes it with a stack of multi-scale residual blocks, reduces the temporal dimension through global pooling, and produces a two-class output: real or fake.

The key architectural choice across all three stages is that the two pre-trained encoders are never fine-tuned. Their weights are frozen throughout. The only components that are trained are the fusion layer and the Nes2Net backbone. This keeps the number of trainable parameters small, prevents catastrophic forgetting, and makes it feasible to train the full system in a small number of epochs.

4.2 Audio Signal Preprocessing

Before feature extraction, each audio clip goes through the same normalization pipeline, applied uniformly across all source datasets.

Step	Description
Channel reduction	Multi-channel recordings are averaged into a single mono waveform.
Sample rate normalization	Every waveform is resampled to 16,000 Hz using Torchaudio.
Fixed-length normalization	Each waveform is normalized to exactly 4 seconds, equal to 64,000 samples.
Padding-validity mask	A frame-level binary mask marks real audio frames as 1 and padded frames as 0.

Channel reduction

If a recording contains more than one channel, the channels are averaged into a single mono waveform. This matches the expected single-channel input format of the pre-trained encoders used in this project.

Sample rate normalization

Every waveform is resampled to **16,000 Hz** using Torchaudio. This is the expected input rate for both WavLM and the Whisper encoder.

Fixed-length normalization

To ensure consistent input dimensions across all samples, each waveform is normalized to exactly **4 seconds (64,000 samples)**.

Clips longer than 4 seconds are randomly cropped. A start offset is drawn uniformly at random from the valid range, and 64,000 consecutive samples are taken from that position. This random crop acts as a mild data augmentation. Clips shorter than 4 seconds are zero-padded at the end to reach the target length.

Padding-validity masking

For each clip, we record the ratio of real (non-padded) samples to total length. This ratio is used to build a frame-level binary mask after feature extraction. Frames corresponding to real audio are marked **1**, while frames corresponding to padding are marked **0**. The mask is stored alongside the features and can be passed to the classifier to reduce attention to padding artifacts.

4.3 Acoustic and Semantic Feature Extraction: WavLM and Whisper

Rather than extracting features on the fly during training, both WavLM and Whisper representations are computed once for all clips and saved as PyTorch tensor files (`.pt`).

Each saved file contains:

- WavLM feature sequence
- Whisper encoder feature sequence, temporally aligned to WavLM
- WavLM and Whisper padding-validity masks
- Integer label: `0` for real, `1` for fake
- Metadata (when enabled in the extraction script): source path, split name, class name, sample rate, and duration

This offline approach avoids redundant forward passes through large pre-trained models during training and significantly reduces GPU memory pressure.

WavLM extraction

The preprocessed 16 kHz mono waveform is passed directly into the frozen WavLM encoder, **microsoft/wavlm-base-plus**. The model processes the raw waveform through its CNN feature extractor and Transformer stack, producing a sequence of **768-dimensional** frame-level embeddings at a temporal resolution of one frame per **20 ms**.

For a 4-second clip, this results in a sequence of **200 frames**, which becomes the fixed temporal length used throughout the pipeline.

Whisper extraction

The same waveform is first converted to an **80-band log-Mel spectrogram**, the required input format for the Whisper encoder. The spectrogram is passed through the frozen Whisper encoder, **openai/whisper-small**. The decoder is not used.

The encoder outputs a sequence of **768-dimensional** contextual embeddings that reflect phonetic and prosodic structure rather than raw acoustic signal properties.

Temporal alignment

WavLM and Whisper produce feature sequences of different temporal lengths for the same 4-second input. After extracting both, the Whisper sequence is aligned to the WavLM sequence length. If Whisper produces more frames, it is truncated. If it produces fewer frames, it is zero-padded at the end.

This alignment ensures that the two sequences share a common time axis and can be combined frame-by-frame in the fusion stage.

4.4 Feature Fusion: Combining WavLM and Whisper

The fusion module takes two temporally aligned feature sequences, each of shape $T \times 768$, and combines them into a single sequence of the same shape. This fused representation is then passed to the classifier.

The fusion is implemented as a **learnable weighted sum**. A vector of **768 scalar weights** is maintained for each of the two input sources. Before combining, these weights are passed through a softmax so they sum to 1 across the two sources at every feature dimension.

At each time step, the fused output is a convex combination of the WavLM and Whisper embeddings, where the mixing proportion for each feature dimension is learned during training.

This formulation is intentionally simple:

- It adds only $2 \times 768 = 1,536$ trainable parameters.
- It imposes no structural assumptions about how the two representations should interact.
- It is interpretable, because the learned weights reveal which model the classifier relies on for each part of the feature space.

4.5 Classification Model: Adapting Nes2Net to the Fused Features

The fused $T \times 768$ sequence is passed to the Nes2Net backbone without dimensionality reduction. Each Nes2Net block receives the full 768-dimensional representation and processes it through its nested Bottle2neck structure and SE module. This progressively refines the representation while preserving both fine-grained and coarser-scale information across the temporal dimension.

After the stack of Nes2Net blocks, global average pooling collapses the temporal dimension **T** into a single **768-dimensional** vector that summarizes the clip. This vector is then passed to a linear classification head that produces two logits, corresponding to the real and fake classes.

During training, cross-entropy loss is applied to these logits. At inference time, the argmax of the logits determines the final prediction.

The input dimension of 768 matches the output of both pre-trained encoders exactly, so no projection layer is needed between the fusion module and the classifier. The NES ratio is set to **(8, 8)** and the dilation factor to **2** within the Bottle2neck blocks, matching the configuration from the original Nes2Net paper. A dropout rate of **0.5** is applied within the backbone during training for regularization.

5. Experimental Setup

5.1 Environment and Software Libraries

All experiments were run on a CUDA-enabled GPU, with automatic fallback to CPU when a GPU was unavailable.

The project is implemented in Python using PyTorch as the main deep learning framework, with Torchaudio for audio loading and resampling. We used the Hugging Face Transformers library to load the pre-trained WavLM and Whisper models.

Additional libraries include NumPy and Pandas for data handling, scikit-learn for evaluation metrics, and Matplotlib and Seaborn for result visualization.

5.2 Training Procedure

We trained the model using the Adam optimizer with a learning rate of **1e-4** and a weight decay of **1e-4** for L2 regularization. The loss function used is standard Cross-Entropy Loss, which is well suited for our binary classification task: real versus fake.

To improve training stability, we applied gradient clipping with a maximum norm of **1.0**. We also used a ReduceLROnPlateau scheduler that halves the learning rate when validation loss does not improve for **2 consecutive epochs**.

Training ran for a total of **8 epochs** with a batch size of **16**, and we applied early stopping with a patience of **4 epochs** to avoid overfitting. The best model checkpoint was saved based on the best combination of validation accuracy and validation loss.

To ensure reproducibility, we fixed the random seed to **42** for PyTorch and the Python random module.

5.3 Hyperparameter Tuning

The main architectural hyperparameters of the Nes2Net classifier were kept consistent with the original design.

Hyperparameter	Value
Input feature dimension	768
NES ratio	(8, 8)
Dilation factor	2
Dropout rate	0.5
Pooling	Global average pooling
Audio sample rate	16,000 Hz
Audio duration	4 seconds
Samples per clip	64,000
Temporal sequence length	200 frames
Fusion type	Learnable weighted sum
WavLM model	microsoft/wavlm-base-plus
Whisper model	openai/whisper-small

The fusion module uses a learnable weighted sum with softmax normalization over the two feature sources, allowing the model to learn how much to rely on WavLM versus Whisper at each feature dimension.

5.4 Evaluation Metrics

We evaluated the model using several complementary metrics.

Metric	Purpose
Accuracy	Measures the overall percentage of correctly classified samples.
Cross-Entropy loss	Tracks convergence and helps detect overfitting.
Confusion matrix	Shows true positives, false positives, true negatives, and false negatives.
Precision	Measures how reliable positive predictions are.
Recall	Measures how many samples of a class are correctly detected.
F1-score	Harmonic mean of precision and recall.
Equal Error Rate (EER)	Measures the point where false acceptance and false rejection rates are equal.

Because Dataset 1 is perfectly balanced and Dataset 2 is near-balanced (with a very small class skew), accuracy serves as a reliable primary metric in both experiments. However, accuracy alone is not sufficient. The confusion matrix, precision, recall, and F1-score reveal whether the model tends to miss fake samples or incorrectly flag real ones.

All metrics are computed after running inference in evaluation mode, with dropout disabled and no gradient updates, across the full test set.

6. Results and Discussion

6.1 Baseline Model Performance: WavLM + Nes2Net

The baseline configuration was designed as an acoustic-only setting, where WavLM representations are directly processed by the Nes2Net classifier.

This provides a controlled reference for the study by keeping the classification objective unchanged, while relying solely on signal-level information. As such, the baseline serves as the primary comparison point for assessing the added value of multimodal integration in the proposed framework, especially when discussing stability and generalization trends.

6.2 Enhanced Model Performance: WavLM + Whisper + Nes2Net

The enhanced configuration extends the baseline by integrating complementary representations from WavLM and Whisper prior to classification. By combining these sources at feature level, the model can jointly consider low-level acoustic signatures and higher-level linguistic patterns.

In this setting, the model benefits from both acoustic cues and higher-level phonetic-linguistic structure, with fusion performed through a learnable weighted mechanism before Nes2Net prediction. The adaptive weighting allows the network to emphasize whichever representation is more informative for each input condition.

Compared with the acoustic-only setup, this combined representation yields stronger and more stable performance behavior, supporting the central hypothesis that semantic-acoustic fusion improves robustness in deepfake speech detection. The observed behavior is consistent with the design goal of reducing dependence on any single cue family.

6.3 Ablation Study: Relative Contribution of Whisper Integration

The ablation logic in this project compares two aligned settings:

Setting	Feature source	Classifier
Baseline	WavLM only	Nes2Net
Proposed model	WavLM + Whisper	Nes2Net

Because both settings keep the same classifier family and decision objective, the main changing factor is the addition of Whisper-derived information. This makes it possible to attribute the observed robustness gains to multimodal feature integration rather than to a different classification backend, which keeps the comparison methodologically clean.

In practical terms, the ablation analysis supports our core claim that adding semantic-phonetic context improves the model's ability to handle difficult cases that are less separable from acoustic cues alone. It also clarifies that the improvement is not merely a byproduct of model size, since the classification head remains conceptually consistent.

6.4 Error Analysis: Strengths and Weaknesses Across Spoofing Types

In Dataset 1, the final test run shows a symmetric error profile, with the same number of false positives and false negatives. In Dataset 2, the profile is slightly asymmetric (FP=77, FN=38), but still strongly balanced relative to the dataset size. Together, these outcomes indicate that the model does not strongly favor one class over the other across either experimental configuration.

Across both experiments, most predictions are highly confident, while the mistaken cases are concentrated around more ambiguous examples, including a subgroup near the decision boundary and another subgroup with high-confidence errors. This suggests that some errors come from genuinely borderline samples, while others likely reflect hard artifacts that still confuse both feature branches.

This pattern suggests two practical directions for improvement:

- 1. **Calibration-oriented techniques** to reduce overconfident mistakes.
- 2. **Targeted data expansion** for edge cases that remain difficult under both acoustic and semantic evidence.

Although the available exported files do not include explicit attack-type tags per sample, the current analysis still provides a useful diagnostic view of where the model is already stable and where it can be strengthened.

7. Conclusion and Future Work

7.1 Summary of Achievements in Model Integration

In this project, we implemented and validated a complete audio deepfake detection pipeline based on feature-level integration between two pre-trained backbones, WavLM and Whisper, followed by classification with a Nes2Net-based head. This end-to-end design keeps representation learning and final decision-making tightly coordinated in one workflow while keeping the frozen encoder strategy unchanged across experiments.

To evaluate the approach fairly, we ran two full experiments that match the two dataset configurations defined earlier in the project:

Experiment	Dataset configuration	Key test outcome
Dataset 1	ASVspoof 2019 LA + ASVspoof5 2024	99.18% accuracy on 10,536 test samples
Dataset 2	ASVspoof 2019 LA + ASVspoof5 2024 + Fake-or-Real (for-norm)	99.34% accuracy and 0.60% EER on 17,467 test samples

In **Dataset 1**, FusionGuardNet was trained and evaluated after balanced split preparation (**84,278 train, 10,534 dev, 10,536 test**), with a near-symmetric error profile and only **86 total mistakes** (**TN=5,225, FP=43, FN=43, TP=5,225**). The training history also showed stable convergence over 8 epochs, rising from **92.60%** train accuracy in epoch 1 to **99.49%** in epoch 8, with best dev accuracy of **99.25%**.

In **Dataset 2**, which adds the Fake-or-Real (for-norm) subset and therefore increases diversity and scale, the same architecture maintained strong performance on a larger test set (**17,467 samples**). The model reached **99.34% test accuracy**, **0.60% EER**, and confusion-matrix counts of **TN=8,652, FP=77, FN=38, TP=8,700**, indicating that the integration remains robust under broader and more modern spoofing conditions.

Taken together, these two experiments show that the main achievement is not only combining acoustic and semantic representations in one architecture, but demonstrating that this combination trains reliably and scales well from the first configuration to the larger mixed-source configuration.

7.2 Suggestions for Improvement and Future Research

For future work, the first priority is to run a fully documented baseline suite using the same reporting format. This should include both **WavLM-only** and **Whisper-only** settings, so the contribution of each branch can be compared directly. A consistent baseline package will make future comparisons clearer and strengthen the reproducibility of conclusions.

Another important step is broader robustness evaluation on additional datasets and unseen generation settings, to check how well the model generalizes outside the current benchmark. This is especially important for understanding whether performance remains stable under distribution shifts and new synthesis artifacts.

On the modeling side, it is worth testing richer fusion strategies, such as attention-based fusion, and calibration methods that can reduce confident mistakes on borderline samples. These extensions could improve both detection reliability and confidence quality in challenging decision regions.

From a deployment perspective, improving runtime and memory efficiency can make the system more suitable for near-real-time screening. Lightweight inference behavior is likely to be a key requirement for practical integration into monitoring pipelines.

Finally, adding interpretability analysis for time-frequency regions can make decisions easier to explain and trust in practical, high-stakes applications. Better interpretability would also help researchers diagnose remaining failure modes and guide targeted data or architecture improvements.

8. References

1. Liu, T., Truong, D. T., Das, R. K., Lee, K. A., & Li, H. (2025). *Nes2Net: A lightweight nested architecture for foundation model driven speech anti-spoofing*. arXiv preprint arXiv: 2504.05657v2.

2. Chen, S., Wang, C., Chen, Z., Wu, Y., Liu, S., Chen, Z., ... & Wei, F. (2022). *WavLM: Large-scale self-supervised pre-training for full stack speech processing*. IEEE Journal of Selected Topics in Signal Processing, 16(6), 1505–1518.
3. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). *Robust speech recognition via large-scale weak supervision*. In International Conference on Machine Learning (pp. 28448–28481). PMLR.
4. Kinnunen, T., Yamagishi, J., Todisco, M., Delgado, H., Sahidullah, M., Wang, W., Evans, N., & Lee, K. A. (2019). *ASVspoof 2019: Future horizons in spoofed and fake audio detection*. In Proc. Interspeech 2019 (pp. 1008–1012).
5. Wang, X., Tak, H., Patino, J., Todisco, M., Nautsch, A., Yamagishi, J., & Evans, N. (2024). *ASVspoof 5: Crowdsourced data, deepfakes, and adversarial attacks at scale*. arXiv preprint arXiv:2408.08703.
6. Kaggle. (n.d.). *Fake or real audio dataset*. <https://www.kaggle.com/datasets/mohammedabdeldayem/the-fake-or-real-dataset>